

Bridge | Decouple an abstraction from its implementation — two class trees joined by composition, each free to vary.

 DIVE 01 · 10 km
Airplane porthole
The big picture & contours

→

 DIVE 02 · 1 km
Train window
Theses, examples, terms

→

 DIVE 03 · 100 m
Park walk
How the pieces relate

→

 DIVE 04 · 1 cm
Microscope
Exact wording & gotchas

 Airplane porthole

DIVE 01 · ORIENTATION

▼ 10 km

Bridge splits an abstraction from its implementation into two class trees — joined by composition, so each side can vary on its own.

THE CONTOURS

• Two hierarchies, not one
abstraction and implementation vary separately

• Composition over inheritance
the abstraction holds an implementor, doesn't extend it

• Kills the class explosion
— M shapes × N renderers becomes M + N, not M × N

USE IT WHEN — GoF applicability

 Both sides must vary
you want to extend abstraction and implementation independently by subclassing

 Avoid a permanent bind
the implementation must be selectable or switchable at runtime

📦 Structural family — Bridge separates abstraction from implementation up front; Adapter retrofits a mismatched interface; Strategy looks identical in code but swaps an algorithm, not a platform.

↓ DIVE DEEPER

 Train window

DIVE 02 · KEY OBJECTS

▼ 1 km

THE THREE PARTS

Abstraction
high-level interface; holds a reference to an Implementor

Implementor
the low-level interface — different from Abstraction's

ConcreteImplementor
a specific platform doing the primitive operations

IN THE WILD

 JDBC
one app API bridged to per-vendor DB drivers

 SLF4J logging
a facade bridged to log4j / logback backends

 AWT component peers
Java widgets bridged to native peers

 Remote × Device
Head First example — one remote drives any device

<> Abstraction.operation() → impl.operationImpl() — the bridge crossing from one tree to the other.

GoF intent: “Decouple an abstraction from its implementation so that the two can vary independently.”

↓ DIVE DEEPER

 Park walk

DIVE 03 · CONNECTIONS

▼ 100 m

Abstraction
high-level API

→

holds an Implementor
the bridge

→

impl.operationImpl()
delegate down

→

✓ ConcreteImplementor
platform work

WHY IT FOLLOWS

→ Reference, not inheritance ⇒ the abstraction composes an implementor instead of extending it

→ Two interfaces ⇒ Abstraction (high-level) and Implementor (primitive ops) evolve apart

→ Swap the implementor ⇒ change platform at runtime without touching the abstraction

🔗 Resembles Adapter, but Bridge is planned up front while Adapter is retrofitted; an Abstract Factory can create the right abstraction–implementor pairs; Strategy shares the shape but swaps behavior, not platform.

STRUCTURE · UML

Abstraction

- impl: Implementor

+ operation(): void

Abstraction.operation() forwards to impl.operationImpl(). RefinedAbstraction and ConcreteImplementor extend each tree independently.

WHAT IT BUYS YOU

✓ Decouples interface & impl — implementation isn't bound to the abstraction at compile time

✓ Both sides extensible — add abstractions or implementations independently

✓ Hides impl from clients — clients only ever see the abstraction's interface

✓ Avoids class explosion — M × N subclasses collapse to M + N classes

✓ Runtime configuration — pick or switch the implementor while the app runs

↓ DIVE DEEPER

 Microscope

DIVE 04 · DETAILS

▼ 1 cm

Definition: “Decouple an abstraction from its implementation so that the two can vary independently.”

— GoF, “Design Patterns”, 1994

THE CODE

```
interface Renderer { circle(r): void; }
class Shape {           // Abstraction
  constructor(protected r: Renderer) {}
}
class Circle extends Shape {
  draw() { this.r.circle(10); }
}
// same Circle, two implementations:
new Circle(new SvgRenderer());
new Circle(new CanvasRenderer());
```

COMPARE THE ALTERNATIVES

Aspect	Bridge	Adapter	Strategy	Abstract F.
Two parallel hierarchies	✓	✗	✗	~
Designed up front	✓	✗	✓	✓
Varies implementation	✓	~	✗	✓
Varies an algorithm	✗	✗	✓	✗
Composition-based	✓	~	✓	✓

Best fit → Bridge: split abstraction from platform · Adapter: fix a mismatch · Strategy: swap an algorithm · Abstract Factory: build matching families. (“~” = partial.)

INTERVIEW PITFALLS

Q “Bridge vs Adapter?”
Bridge is designed in beforehand so abstraction and implementation vary independently; Adapter is added after the fact to make an existing class fit.

Q “Bridge vs Strategy?”
The code looks the same — composition + delegation. Bridge separates an abstraction from its platform/impl; Strategy swaps an interchangeable algorithm.

Q “What problem does it solve?”
The M × N subclass explosion: two independent dimensions get separate hierarchies, giving M + N classes instead of M × N.

Q “Where is the bridge?”
The reference the Abstraction holds to an Implementor — calls cross from the abstraction tree to the implementation tree there.

VARIANTS

Classic Bridge
abstraction holds an implementor reference — GoF form

Refined abstractions
subclass the abstraction side independently

Pimpl idiom
C++ “pointer to implementation” — a Bridge in disguise

Driver model
one API, many swappable backend drivers

DI-injected impl
the implementor is injected, not newed inside

Strategy-like
same shape; Bridge varies platform, Strategy varies algorithm

NUANCES & GOTCHAS

• vs Adapter — Bridge is planned to vary two sides; Adapter is added later to fix a mismatch

• vs Strategy — structurally identical; Bridge separates platform / impl, Strategy swaps an algorithm

• Two interfaces, not one — Abstraction and Implementor are deliberately different shapes

• Combats M × N classes — independent trees turn a product of subclasses into a sum

• Who picks the impl? — a factory, DI, or the abstraction's constructor wires the pair

• Over-engineering risk — with only one implementation it's needless indirection

WATCH OUT

⚠ **Anti-pattern:** Skip it until a second implementation is real — with one platform the extra hierarchy is pure overhead. If only behavior varies (not a platform / impl), reach for Strategy instead.

⚡ **Modern take:** in TS, inject the implementor through the constructor (DI) and program the abstraction to its interface — that is a Bridge, whether or not anyone names it.